

Plan de Dimensionamiento, Alcance Individual y Metodología (AE2)

Proyecto: Ombú - Cervecería de Barrio (Self-Service Tap Wall NFC)

Asignatura: Paradigmas y Lenguajes de Programación III (UCP - FAITA)

Actividad: Actividad de Evaluación N° 2 (AE2) - Instancia Individual

Integrantes:

- **Lucrecia Sabrina Mencia** (Módulo: *Lecturas NFC y Despachos*)
- **Luciano Rubén Chesani** (Módulo: *Barriles y Control de Stock*)

Docente a cargo: Prof. Carlos Emiliano Pereyra

Ventana de ejecución: Sprint intensivo de 1 semana (7 días)

1. Introducción y Marco de la Actividad

La **Actividad de Evaluación 2 (AE2)** es una instancia formativa e **individual** de evolución y profundización arquitectónica sobre la solución iniciada en AE1.

El objetivo pedagógico es **evolucionar una solución compartida dividiendo responsabilidades en dos módulos complementarios de alcance Full-Stack (Frontend y Backend)**, garantizando:

- Trazabilidad individual de commits, tareas e issues.
 - Límites claros de responsabilidad (*Bounded Contexts*) y propiedad de datos (*Database Ownership*).
 - Comunicación síncrona mediante contratos REST explícitos.
 - Comunicación asíncrona desacoplada mediante **RabbitMQ** (Productor / Consumidor).
 - Gestión de estado efímero y caché con **Redis** (TTL, invalidación, atomicidad).
 - Tratamiento de **concurrency, consistencia e idempotencia**.
 - Generación de artefactos requeridos (Ticket/Comprobante PDF).
-

2. Nomenclatura de Ramas y Entorno de Trabajo

Para garantizar la organización del repositorio y la trazabilidad modular:

- **Punto de partida común:** Tag base congelado de AE1 (`v1.0-ae1`).
- **Rama Lucrecia (Módulo NFC):**

`ae2/lecturas-nfc`

- **Rama Luciano (Módulo Stock):**

`ae2/barriles-stock`

- **Regla estricta:** Está terminantemente prohibido realizar commits directos sobre la rama `main` . Toda evolución se realiza en la rama del módulo correspondiente y se integra al finalizar la actividad mediante Pull Request.
-

3. Delimitación de Alcance Full-Stack por Integrante

Ambos integrantes desarrollarán tanto la **capa de cliente (Frontend)** como la **capa de servicios (Backend y Persistencia)** de sus respectivos dominios.

CERVECERÍA OMBÚ TAP WALL	
LUCRECIA SABRINA MENCIA Rama: ae2/lecturas-nfc	LUCIANO RUBÉN CHESANI Rama: ae2/barriles-stock
FRONTEND: • Terminal / Simulador de Autoservicio • Selección de formato y lectura NFC • Feedback visual con cuenta regresiva TTL • Descarga del comprobante en PDF BACKEND & DATOS: • API REST de Tarjetas y Despachos • [Redis] Sesión temporal de grifo (TTL) • [Idempotencia] Idempotency-Key en cobro • [RabbitMQ] Productor: DespachoRealizado • Generación de comprobante en PDF	FRONTEND: • Panel de Monitoreo de Barriles y Stock • Barras de nivel en tiempo real • Alertas de stock crítico y reposición • Modal interactivo de cambio de barril BACKEND & DATOS: • API REST de Barriles y Asignación • [Redis] Caché de alta lectura de stock • [Concurrencia] Bloqueo transaccional RN03 • [RabbitMQ] Consumidor: DespachoRealizado • Descuento de litros y estado Sin Stock

A. Módulo de Lucrecia: Lecturas NFC y Despachos (ae2/lecturas-nfc)

1. Capa Frontend (Cliente)

- **Simulador de Terminal NFC:** Vista interactiva en la interfaz web de Ombú que permite simular la aproximación de una tarjeta NFC (**UID**) a una canilla seleccionada.
- **Flujo de Autoservicio:**
 1. El cliente pasa la tarjeta: el frontend muestra el saldo disponible y el titular.
 2. Muestra un temporizador regresivo visual sincronizado con el TTL de Redis (45s) indicando: "Grifo habilitado: sírvase antes de que expire la sesión".
 3. Selección de formato (**Media Pinta - 250ml** , **Pinta - 500ml** , **Litro - 1000ml**).
 4. Confirmación del servido y botón inmediato para **Descargar Comprobante PDF**.

2. Capa Backend y Persistencia

- **API REST (/api/nfc y /api/despachos):**
 - **POST /api/nfc/autenticar** : Valida tarjeta, saldo y crea sesión efímera en Redis.
 - **POST /api/despachos** : Procesa el débito del cliente y registra el despacho.
- **Uso de Redis (Estado Efímero):**
 - Clave: **nfc:sesion:{uid_tarjeta}** con valor { **id_canilla**, **saldo_disponible** }.
 - **TTL obligatorio de 45 segundos.** Si el cliente no activa el grifo, la clave expira en memoria y cualquier intento posterior es rechazado sin debitar fondos.
- **Manejo de Idempotencia y Concurrencia:**
 - Implementación de la cabecera **Idempotency-Key** (o hash **uid + timestamp + canilla**).
 - Si la terminal o el usuario reenvía la petición por error de red o múltiple toque accidental, el backend reconoce la clave existente en Redis/DB y devuelve la misma respuesta sin volver a cobrar.
- **Uso de RabbitMQ (Productor de Mensajería):**
 - Al completar con éxito la transacción en SQL Server, publica de forma no bloqueante el evento **DespachoRealizado** hacia el exchange **ombu.eventos** .
- **Artefacto PDF:**
 - Microservicio o función utilitaria con **pdftkit** que construye y emite el ticket digital de consumo.

B. Módulo de Luciano: Barriles y Control de Stock (`ae2/barriles-stock`)

1. Capa Frontend (Cliente)

- **Dashboard Administrativo de Barriles y Stock:** Vista especializada dentro del sistema Ombú para el personal de barra y depósito.
- **Visualización de Volumen en Tiempo Real:**
 - Tarjetas de barriles conectados mostrando porcentaje de llenado mediante barras de progreso dinámicas.
 - Badges de alerta reactiva: `Óptimo` (>20L), `Nivel Bajo` (<10L) y `Agotado / Sin Stock` (0L).
- **Modal de Gestión y Conexión de Barriles:** Formulario para conectar un barril de depósito a una canilla física vacía, o realizar el recambio.

2. Capa Backend y Persistencia

- **API REST (`/api/barriles` y `/api/stock`):**
 - `GET /api/stock/resumen` : Consulta rápida del volumen de todas las canillas.
 - `POST /api/barriles/conectar` : Asocia un barril a una canilla física.
 - `DELETE /api/barriles/:id` : Aplica *Soft Delete* pasando el estado a `'Retirado'` .
- **Concurrencia y Transaccionalidad (Regla RN-03):**
 - Implementar aislamiento transaccional (`SERIALIZABLE` o `BEGIN TRANSACTION` con `UPDLOCK`) en SQL Server para impedir que dos operadores asignen concurrentemente el mismo barril o conecten dos barriles a la misma canilla.
- **Uso de Redis (Caché de Alta Lectura):**
 - Clave: `stock:muro:resumen` .
 - Los dashboards consultan el stock caliente desde Redis en `<5\text{ms}`\$. Se invalida automáticamente cuando se procesa un despacho o se conecta un nuevo barril.
- **Uso de RabbitMQ (Consumidor Asíncrono):**
 - Servicio trabajador (*worker*) que escucha permanentemente la cola `stock.despachos.queue` .
 - Ante la llegada de `DespachoRealizado` :
 1. Descuenta los mililitros consumidos del campo `litros_restantes` de la tabla `Barril` .
 2. Si los litros llegan a 0, actualiza el estado de la `Canilla` a `'Sin Stock'` .
 3. Invalida la clave de caché en Redis.
- **Idempotencia del Consumidor:** Verifica que el `idDespacho` no haya sido procesado previamente ante un eventual reenvío de RabbitMQ.

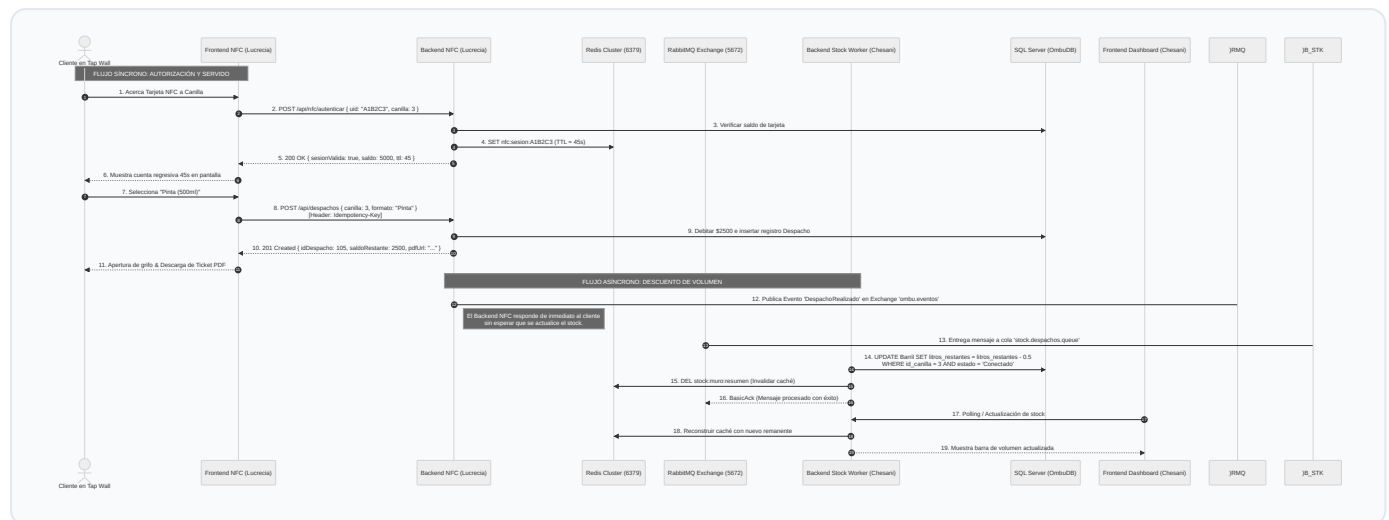
4. Contrato de Integración y Arquitectura de Comunicación

4.1 Principio de Propiedad de Datos (*Database Ownership*)

Para satisfacer la rúbrica de arquitectura distribuida (Criterios 3 y 8):

- **Módulo NFC (Lucrecia)** es dueño exclusivo de: `Cliente` , `TarjetaNFC` , `Recarga` , `Despacho` .
- **Módulo Stock (Chesani)** es dueño exclusivo de: `Barril` , `Canilla` , `Cerveza` , `EstiloCerveza` .
- **Prohibición estricta:** Ningún módulo realiza `JOIN` ni consultas `SELECT` / `UPDATE` sobre las tablas del otro módulo. Toda sincronización de estado se realiza a través de eventos en RabbitMQ o consultas REST expuestas con contratos formales.

4.2 Diagrama Integral de Secuencia y Comunicación



4.3 Especificación del Contrato Asíncrono (RabbitMQ Event Schema)

- **Exchange:** `ombu.eventos` (Tipo: `topic`)
- **Routing Key:** `despacho.confirmado`
- **Queue:** `stock.despachos.queue` (Durable: `true`)
- **Formato del Mensaje:** JSON estructurado

JSON Payload:

```

{
  "eventId": "evt-77f9c2d1-4e8b-4a56-91e2-b88301fa3201",
  "eventType": "DespachoRealizado",
  "occurredAt": "2026-09-22T20:15:30.120Z",
  "version": "1.0",
  "idempotencyKey": "idem-nfc-tap-884920-c3",
  "producer": "modulo-nfc-despachos",
  "data": {
    "idDespacho": 105,
    "idCanilla": 3,
    "volumenLitros": 0.500,
    "formato": "Pinta",
    "mililitros": 500,
    "importeDebitado": 2500.00,
    "idTarjeta": 14,
    "idCliente": 2
  }
}
  
```

Política de Resiliencia y Reintentos:

1. **Reconocimiento manual (`manual Ack`):** El consumidor de stock no envía `ack` hasta que la transacción en la base de datos se confirme con éxito.
2. **Idempotencia en Consumidor:**

```
-- El consumidor valida si el despacho ya fue aplicado al barril
IF NOT EXISTS (SELECT 1 FROM DespachoProcesadoStock WHERE id_despacho = @idDespacho)
BEGIN
    UPDATE Barril SET litros_restantes = litros_restantes - @volumenLitros WHERE id_canilla = @idCanilla
    INSERT INTO DespachoProcesadoStock (id_despacho, fecha_procesado) VALUES (@idDespacho, GETDATE())
END
```

1. **Dead Letter Queue (DLQ):** Tras 3 reintentos fallidos, el mensaje se redirige a `stock.despachos.dlq` para inspección sin detener el flujo general.

5. Metodología de Trabajo: Sprint de 1 Semana (7 Días)

Dado el plazo estricto de **una semana**, se aplica un marco **Kanban ágil con entregas continuas diarias**:

Cronograma Día a Día

CRONOGRAMA DE 7 DÍAS - SPRINT AE2		
DÍA	LUCRECIA (ae2/lecturas-nfc)	LUCIANO (ae2/barriles-stock)
Día 1	Setup rama ae2/lecturas-nfc. Docker Compose (Redis + RabbitMQ). Contrato OpenAPI de NFC.	Setup rama ae2/barriles-stock. Docker Compose (Redis + RabbitMQ). Contrato OpenAPI de Barriles.
Día 2	Backend NFC: autenticar tarjeta y sesión efímera en Redis (TTL 45s).	Backend Stock: CRUD de barriles y Soft Delete ('Retirado').
Día 3	Frontend NFC: vista de terminal, simulación de tap y cuenta regres.	Backend Stock: transacciones concurrentes de asignación (RN-03).
Día 4	Backend NFC: débito con clave de idempotencia y publicación Rabbit.	Backend Stock: consumidor RabbitMQ para procesar evento DespachoRealizado.
Día 5	Frontend NFC: feedback de servido y generación de Comprobante PDF.	Frontend Stock: dashboard de barriles, barras de progreso y alerta 'Sin Stock'.
Día 6	Pruebas de integración cruzada: paso de tarjeta en NFC -> descuento en Stock. Validación de idempotencia, reconexiones y tolerancia a fallos.	
Día 7	Cierre de Portafolio Digital, bitácoras individuales, documentación README y preparación del paquete de entrega individual en campus virtual.	

6. Matriz de Rúbrica y Evidencias a Presentar

Criterio Rúbrica AE2	Evidencia Lucrecia (ae2/lecturas-nfc)	Evidencia Luciano (ae2/barriles-stock)
1. Trazabilidad	Rama ae2/lecturas-nfc , commits #issues	Rama ae2/barriles-stock , commits #issues
2. Requerimientos	Terminal NFC, autorización y cobro	Gestión de barriles, conexión y mermas
3. Arquitectura	Desacoplamiento total de clientes/ventas	Desacoplamiento total de inventario/frío
4. Contratos REST	OpenAPI de terminal NFC y despachos	OpenAPI de administración de barriles/stock
5. RabbitMQ	Productor del evento DespachoRealizado	Consumidor con ack y descuento de stock
6. Redis	Sesión efímera de grifo con TTL 45s	Caché caliente stock:muro:resumen
7. Concurrencia	Idempotencia en cobro (<i>Idempotency-Key</i>)	Bloqueo transaccional de canilla única (RN-03)
8. Datos	Dueña de TarjetaNFC , Cliente , Despacho	Dueño de Barril , Canilla , Cerveza
9. Artefactos	Generación de Ticket / Comprobante PDF	Panel visual interactivo de barras de nivel
10. Testing	Pruebas unitarias de cobro y expiración TTL	Pruebas de concurrencia y descuento de litros

Documento aprobado como guía oficial de trabajo individual y coordinación para AE2.